

TP1 - Projeto de Execução Dinâmica de Processos

O presente trabalho tem por objetivo explorar temas referentes ao escalonamento, preempção e troca entre processos que utilizam um dado processador. É previsto o desenvolvimento de um ambiente que empregue uma política de **Escalonamento por Filas Múltiplas com Realimentação (MLFQ)**, bem como gerencie a inclusão, transição de estados e remoção de processos que ocupam a CPU. A carga de processos deverá ser realizada a partir de programas escritos em uma linguagem assembly hipotética

1. Descrição de programas e características de execução e ocupação

O usuário deverá ser capaz de descrever pequenos programas a serem executados pelo ambiente. O ambiente de execução é baseado em acumulador. Assim, para a execução de um programa, dois registradores estão presentes: **(i)** o acumulador (acc) onde as operações são realizadas e **(ii)** o ponteiro da instrução em execução (pc). A linguagem de programação hipotética a ser empregada é definida na Tabela 1.

Tabela 1 - Mnemônicos e funções associadas

<u>Categoria</u>	<u>Mnemônico</u>	<u>Função</u>
Aritmético	ADD op1	acc=acc+(op1)
	SUB op1	acc=acc-(op1)
	MULT op1	acc=acc*(op1)
	DIV op1	acc=acc/(op1)
Memória	LOAD op1	acc=(op1)
	STORE op1	(op1)=acc
Salto	BRANY label	pc ← label
	BRPOS label	Se acc > 0 então pc <- op1
	BRZERO label	Se acc = 0 então pc <- op1
	BRNEG label	Se acc < 0 então pc <- op1
Sistema	SYSCALL index	Chamada de sistema

As instruções da categoria aritmético podem operar em modo de endereçamento imediato ou direto. No modo de endereçamento imediato, o operando é precedido pelo caractere # para indicar que *op1* é um valor numérico constante (ex: SUB #1). No modo direto *op1* representa o nome da variável na área de dados cujo valor deve ser capturado/escrito (ex: LOAD variable).

Assim como assumido para as instruções aritméticas, a instrução de leitura da memória de dados pode ser realizada tanto em modo imediato quanto direto (ex: LOAD #1 ou LOAD A). Já o comando de escrita dá suporte apenas ao modo direto de operação (ex: STORE A).

Em relação a instrução de operação com o sistema, deverá ser possível associar 3 tipos de pedidos, representados pelos valores numéricos '0', '1' e '2'. Uma chamada de sistema com o valor '0' (zero) realiza um pedido de encerramento do programa (*halt*). Uma chamada de sistema com o valor '1' (um) realiza um pedido de impressão de um valor inteiro na tela. Uma chamada de sistema com o valor '2' (dois) realiza um pedido de leitura de um valor inteiro, que deverá ser feito via teclado. Às chamadas de sistema '1' e '2' provocam a transição do processo para o estado bloqueado por exatamente 3 Unidades de Tempo (UTs).

1 .code	# Define o início da área de código
2 LOAD variable	# Carrega em acc o conteúdo de variable
3 ponto1: SUB #1	# Subtrai do acc um valor constante (i.e. 1)
4 SYSCALL 1	# Imprime na tela o conteúdo do acumulador
5 BRPOS ponto1	# Caso acc>0 deve voltar à linha marcada por "ponto1"
6 SYSCALL 0	# Sinaliza o fim do programa
7 .endcode	# Define o final da área de código
8 .data	# Define o início da área de dados
9 Variable 3	# Conteúdo da posição 1 da área de dados é 3
10.enddata	# Define o final da área de dados

Figura 1 - Código exemplo.

Como finalização da descrição dos processos, devem ser assumidas as seguintes características.

- Ocupação de memória
 - Cada instrução e cada variável ocupam uma posição da memória
 - O número de posições de um programa é então definido pelo número de instruções somado ao número de variáveis;
 - A organização de ocupação da memória por parte dos processos está fora do escopo deste trabalho;
- Os valores atribuídos às variáveis ou às instruções que operam de modo imediato podem assumir valores tanto positivos quanto negativos
- Cada instrução leva 1 unidade de tempo (UT) para ser executada pelo processador
- O tempo de troca de contexto deve ser desconsiderado

2. Política de escalonamento: Filas Múltiplas com Realimentação (MLFQ)

O sistema gerencia duas filas de processos com prioridades globais distintas:

Prioridade Global: Fila 0 (Alta Prioridade) > Fila 1 (Baixa Prioridade)

Se a **Fila 0** possuir qualquer processo no estado *Pronto*, a CPU é atribuída a ela. Um processo em execução na **Fila 1** sofrerá **preempção imediata** caso um processo entre na Fila 0 (seja por nova admissão ou por retorno de bloqueio de I/O).

Regras de Funcionamento das Filas

1. Fila 0: Alta Prioridade (Round Robin Puro):

- **Entrada:** Todos os processos novos admitidos e processos retornando do estado *Bloqueado* (I/O) entram obrigatoriamente no final da Fila 0.
- **Política:** Round Robin (RR) tradicional com Quantum = 2 UTs.
- **Ordenação:** Ordem de chegada (FIFO).
- **Deslocamento:** Se o processo esgotar seu quantum (2 UTs) sem finalizar nem solicitar I/O, ele sofre preempção e é rebaixado para a **Fila 1**.

2. Fila 1: Baixa Prioridade (Round Robin com Prioridades):

- **Política:** Round Robin com Prioridade Estática (Quantum = 4 UTs).
- **Ordenação:** Os processos são ordenados dentro da fila conforme o seu nível de prioridade (ex: valores de 1 a 5, onde maior valor indica maior prioridade na fila).
- **Desempate:** Em caso de processos com o mesmo nível de prioridade, aplica-se a ordem de chegada (FIFO).
- **Comportamento no Estouro do Quantum:** Se o processo consumir os 4 UTs, ele volta para a Fila 1, posicionado ao final do seu respectivo grupo de prioridade.
- **Preempção por Interrupção:** Se for preemptado por um evento que popular a Fila 0, ao retornar para o estado *Pronto*, ele deve voltar ao topo do seu grupo de prioridade na Fila 1 para manter o tempo restante de execução.

Para o algoritmo de escalonamento implementado, deve-se assumir que, supondo que uma tarefa A venha a ser removida do processador para que uma tarefa B o ocupe, quando da retomada da tarefa A, esta deve seguir sua execução do último ponto de parada. O tempo necessário para troca de contexto deve ser desconsiderado para o presente trabalho.

3. Interface da aplicação

O ambiente deve permitir carregar processos via arquivo de configuração ou interface interativa:

- **Parâmetros de Entrada por Processo:** Nome do processo, instante de carga (arrival time), prioridade estática (1 a 5) e arquivo contendo o código assembly hipotético.
- **Saída Esperada (Monitoramento a cada UT):**
 - Estado atual de cada processo (*Pronto, Executando, Bloqueado, Finalizado*).
 - Representação visual ou textual da ocupação da CPU ao longo do tempo (Diagrama de Gantt).
 - Conteúdo da **Fila 0**, **Fila 1** e da **Lista de Bloqueados** em cada instante de tempo.
 - Estatísticas Finais: Tempo de Retorno (*Turnaround*) individual e Tempo Médio de Espera na Fila de Prontos.

4. Informações adicionais

O trabalho deverá ser realizado em grupos de 3 alunos (obrigatoriamente). Deverá ser entregue o código fonte do programa desenvolvido, bem como um manual do usuário em formato PDF contendo as explicações de como compilar e executar o programa, além do detalhamento de 2 casos de teste simulados pelo grupo. A linguagem de programação utilizada para desenvolver o trabalho é de escolha do grupo, desde que seja possível compilar e executar o programa no ambiente computacional disponível nos notebooks disponibilizados pela Escola.

A data de entrega está prevista para o dia 24/09/2026, até às 17hs15min. As apresentações serão realizadas pelos alunos a partir do material postado no Moodle.

Cada aluno deverá assinalar no Moodle o grupo ao qual pertence e **um integrante** do grupo deverá escolher **um horário** para o grupo apresentar o trabalho no dia 24/09/2026 ou no dia 29/09/2026, dentre os horários disponíveis no Moodle.

O trabalho deverá ser entregue no moodle a partir de um arquivo compactado (.zip). O nome do arquivo deverá ser tal que contenha o nome e sobrenome de todos integrantes do grupo. O material postado no moodle é de inteira responsabilidade do aluno. A presença de arquivos corrompidos, que impeçam a avaliação do trabalho pelo professor será considerada como a não entrega do trabalho. Também não serão considerados trabalhos com erro de compilação. Casos em que sejam identificados plágio/cópia ou uso de ferramentas de IA, receberão nota zero.

5. Programas de testes básicos

Esta seção apresenta 2 programas de validação para verificar os requisitos operacionais do trabalho antes da apresentação final.

Configurações Globais:

- Tempo de I/O (SYSCALL 1 e SYSCALL 2): 3 UTs (Unidades de Tempo) fixas.
- Fila 0 (Alta Prioridade): Round Robin puro, Quantum = 2 UTs.
- Fila 1 (Baixa Prioridade): Round Robin com Prioridade Estática, Quantum = 4 UTs.
- Regra de Transição por I/O: Qualquer processo que saia do estado Bloqueado (após cumprir as 3 UTs de I/O) deve retornar obrigatoriamente ao final da Fila 0.

Programa de Teste 1: Testar as operações básicas da CPU, leitura da seção de dados (.data), execução das instruções aritméticas (LOAD, ADD, STORE), bloqueio por I/O de 3 UTs via SYSCALL 1 e finalização limpa via SYSCALL 0

```
.code
    LOAD valor
    ADD #5      # acc = 10 + 5 = 15
    STORE valor
    SYSCALL 1   # Imprime 15 (Bloqueia por 3 UTs no estado BLOQUEADO)
    SYSCALL 0   # Encerra o processo (HALT)
.endcode

.data
    valor 10
.enddata
```

Parâmetros de carga:

- Instante de carga (arrival time): 0 UT
- Prioridade estática (Fila 1): 3

Fluxo no escalonador: $t=0 \rightarrow$ Entra na Fila 0 $\rightarrow$ Transição para Bloqueado (3 UTs) na SYSCALL 1 $\rightarrow$ Retorna para a Fila 0 $\rightarrow$ Executa SYSCALL 0 e encerra.

Programa de Teste 2: Testar a preempção por estouro de Quantum (2 UTs) com rebaixamento para a Fila 1, execução do salto condicional (BRPOS) e a reintegração obrigatória à Fila 0 após cada bloqueio de I/O provocado pela SYSCALL 1

```
.code
    LOAD limite
loop:
    SUB #1      # Decrementa o acumulador (acc = acc - 1)
    STORE temp  # Salva o contador atual em memória
    SYSCALL 1   # Imprime acc e BLOQUEIA por 3 UTs (força retorno à Fila 0)
    LOAD temp   # Recupera o contador para a próxima iteração
    BRPOS loop  # Enquanto acc > 0, continua no laço
    SYSCALL 0   # Encerra o processo (HALT)
.endcode

.data
    limite 3
    temp 0
.enddata
```

Parâmetros de carga:

- Instante de carga (arrival time): 1 UT
- Prioridade estática (Fila 1): 5

Comportamento esperado:

1. Rebaixamento de Fila: Se a execução inicial do laço ultrapassar 2 UTs contínuas de CPU, o processo é movido para a Fila 1.
2. Retorno à Fila 0: Ao invocar a SYSCALL 1, o processo é bloqueado por 3 UTs. Ao concluir a I/O, o escalonador deve inseri-lo obrigatoriamente na Fila 0 (e não na Fila 1).

Saída do Sistema:

Ao cadastrar ambos os programas simultaneamente no ambiente com os tempos de chegada especificados ($t=0$ para P1 e $t=1$ para P2), o log de saída do terminal deve exibir os valores na seguinte ordem temporal:

```
== EXECUTANDO SIMULAÇÃO ==  
[P1] Impressão (SYSCALL 1): 15  
[P2] Impressão (SYSCALL 1): 2  
[P2] Impressão (SYSCALL 1): 1  
[P2] Impressão (SYSCALL 1): 0  
[P1] Finalizado (SYSCALL 0)  
[P2] Finalizado (SYSCALL 0)
```

Linha do Tempo da Simulação Concorrente

- **t=0:** **P1** chega e entra na Fila 0. Executa LOAD valor e ADD #5 (2 UTs, esgotando o Quantum=2).
- **t=1:** **P2** chega e entra no final da Fila 0.
- **t=2:** **P1** sofre preempção e é rebaixado para a Fila 1. **P2** assume a CPU (Fila 0).
- **t=2 a t=3:** **P2** executa LOAD limite e SUB #1 ($acc=2$). Esgota seu Quantum=2 e cai para a Fila 1.
- **t=4:** Fila 0 vazia. **P2** (Prioridade 5) assume a CPU antes de P1 (Prioridade 3) na Fila 1. Executa STORE temp e SYSCALL 1.
- **t=5:** **P2** entra no estado **Bloqueado** por 3 UTs ($t=5, 6, 7$).
- **t=5 a t=6:** **P1** assume a CPU na Fila 1. Executa STORE valor e SYSCALL 1. Imprime **15**.
- **t=7:** **P1** entra no estado **Bloqueado** por 3 UTs ($t=7, 8, 9$).
- **t=8:** **P2** conclui I/O e acorda na **Fila 0** (imprime **2**). Assume a CPU por 2 UTs ($t=8, 9$), executando LOAD temp e SUB #1 ($acc=1$).
- **t=10:** **P1** conclui I/O e acorda na **Fila 0** (atrás de P2 ou P2 estoura quantum). **P2** executa STORE temp e SYSCALL 1 (imprime **1**).
- **t=11:** **P2** entra no estado **Bloqueado** por 3 UTs ($t=11, 12, 13$).
- **t=11:** **P1** (que estava na Fila 0) assume a CPU e executa SYSCALL 0 (Halt). **P1 finaliza em t=11**.
- **t=14:** **P2** conclui I/O e entra na **Fila 0**. Executa LOAD temp, SUB #1 ($acc=0$), STORE temp e SYSCALL 1 (imprime **0**).
- **t=15:** **P2** entra em seu último **Bloqueado** por 3 UTs ($t=15, 16, 17$).
- **t=18:** **P2** acorda na Fila 0, executa SYSCALL 0 (Halt) e encerra. **P2 finaliza em t=18**.

Métrica 1: Tempo de Turnaround Individual

Turnaround = Tempo de Encerramento - Tempo de Chegada

- **P1:**
 - Chegada: $t = 0$ UT
 - Encerramento: $t = 11$ UTs
 - Turnaround-P1 = $11 - 0 = 11$ UTs

- **P2:**

- Chegada: $t = 1$ UT
- Encerramento: $t = 18$ UTs
- Turnaround-P2 = $18 - 1 = 17$ UTs

Métrica 2: Tempo de Espera na Fila de Prontos

Tempo de Espera = Turnaround - Tempo de CPU - Tempo de Bloqueio (I/O)

- **P1:**

- Tempo de CPU: 4 UTs
- Tempo em Bloqueio (1 * SYSCALL 1): 3 UTs
- Espera-P1 = $11 - 4 - 3 = 4$ UTs (*Esperou em $t=1, t=7, t=8, t=9, t=10$*)

- **P2:**

- Tempo de CPU: 8 UTs
- Tempo em Bloqueio (3 * SYSCALL 1): 9 UTs
- Espera-P2 = $17 - 8 - 9 = 0$ UTs

$$\text{Tempo Médio de Espera na Fila de Prontos} = \frac{4 + 0}{2} = 2 \text{ UTs}$$

$$\text{Tempo Médio de Turnaround} = \frac{11 + 17}{2} = 14 \text{ UTs}$$